# ЧТО ПОКАЗЫВАЕТ ОТЧЕТ

Подробное объяснение отчетов EH/Unwind Analyzer: какие поля есть, что они означают и какую информацию из них можно получить

Этот документ описывает текущий отчет проекта, а не будущие желаемые функции. Он сверяется с реальной JSON-схемой docs/report.schema.json, генератором отчетов src/report.c, README.md и демонстрационным reports/demo_ru.json/html. Главная мысль: отчет не является пошаговой эмуляцией RtlVirtualUnwind, но является структурным и эвристическим досье по PE x64 EH/Unwind metadata.

| Источник проверки | Что из него взято |
|---|---|
| docs/report.schema.json | Полный контракт JSON: верхние блоки, обязательные поля, вложенные структуры. |
| src/report.c | Фактическая сериализация JSON, HTML, CSV и Ghidra CSV, русские пояснения findings и manual steps. |
| reports/demo_ru.json/html | Пример реального отчета: какие блоки появляются при запуске с эвристиками. |
| README.md | Назначение форматов: JSON, HTML, CSV, Ghidra CSV и динамический режим. |

Версия формата отчета по текущей схеме: 1.1.0. Проект: EH/Unwind Analyzer 1.0.0.

# 1. Главный смысл отчета

Отчет показывает, как устроены exception/unwind-метаданные PE32+ AMD64 файла и насколько им можно доверять как структурной карте функций, обработчиков и unwind-информации. Он нужен не для того, чтобы заменить реверсера, а чтобы быстро собрать доказательную картину: где лежат важные структуры, какие диапазоны функций описаны, где находятся UNWIND_INFO, какие handler-ы указаны, есть ли chain, какие поля повреждены или выглядят нетипично.

Если сказать совсем строго: отчет отвечает на вопрос “что статически и динамически известно о EH/Unwind metadata этого PE-файла, какие инварианты нарушены и какие места требуют ручной проверки”. Он не отвечает на вопрос “какая инструкция машинного кода точно соответствует каждой UWOP” и не выполняет полноценную симуляцию RtlVirtualUnwind по реальному стеку.

Поэтому отчет важен как рабочая карта для реверсера: он экономит время на первичном разборе PE, .pdata, .xdata, handlers, секций, RVA/file offset и типовых аномалий. Реверсер получает не просто таблицу адресов, а набор связей: функция -> unwind RVA -> секция metadata -> handler -> finding -> severity -> что проверить вручную.

# 2. Какие форматы отчетов есть

| Формат | Как получить | Для чего нужен |
|---|---|---|
| JSON | --json report.json | Главный машинно-читаемый отчет. Подходит для БД, backend, frontend, сравнения запусков, автоматической проверки schema и сохранения evidence. |
| HTML | --html report.html | Человеко-читаемый dashboard. Удобен для защиты, заказчика и ручного анализа: карточки, таблицы, русские пояснения, findings, динамика. |
| CSV | --csv functions.csv | Плоская таблица функций и unwind-сводки. Удобна для Excel, быстрой сортировки, фильтрации и статистики. |
| Ghidra CSV | --ghidra-csv ghidra.csv | Файл для импорта в Ghidra: диапазоны функций, имена sub_XXXXXXXX и комментарии с unwind-сводкой. |
| Console dump | --dump --ops | Быстрый терминальный просмотр RUNTIME_FUNCTION и decoded UWOP-операций. |

Форматы можно комбинировать в одном запуске. Например, один запуск может создать JSON для базы, HTML для человека, CSV для таблицы и Ghidra CSV для дизассемблера.

# 3. Верхний уровень JSON-отчета

JSON-отчет состоит из крупных блоков. Каждый блок отвечает за отдельный слой анализа: входной файл, PE-заголовки, секции, эвристики, динамику, итоговую оценку и список функций.

| Блок | Что показывает | Что можно из него получить |
|---|---|---|
| input | Путь к анализируемому файлу. | Понять, какой именно объект анализировался; связать отчет с загруженным файлом. |
| report_metadata | Версия схемы, версия инструмента, SHA-256, размер, команда запуска, сборка и параметры анализа. | Проверить воспроизводимость: тем ли инструментом, с какими опциями и по какому файлу получен отчет. |
| image_base / size_of_image / size_of_headers | Базовые параметры PE image. | Переводить RVA/VA в правильном контексте; понимать границы загруженного образа. |
| section_alignment / file_alignment | Выравнивание секций в памяти и файле. | Проверять корректность layout и подозрительные нарушения PE-структуры. |

| Блок | Что показывает | Что можно из него получить |
|---|---|---|
| exception_directory | RVA и размер Exception Directory. | Понять, где начинается .pdata/RUNTIME_FUNCTION table и есть ли она вообще. |
| pe_validation | Счетчики структурных проблем PE. | Быстро увидеть, не сломана ли сама карта файла до анализа unwind. |
| sections | Список секций с RVA, raw offset, размером и правами. | Понять, где код, где metadata, где writable/executable области. |
| heuristic_analysis | Счет, confidence, verdict, findings и weakening rules. | Понять, какие признаки аномалий обнаружены и насколько они сильные. |
| dynamic_validation | Результаты Windows dynamic режима, если он запускался. | Понять, были ли исключения, handler hits, VEH/VCH/UEF и runtime function table API calls. |
| assessment | Итоговая корреляционная оценка статики и динамики. | Получить краткий вывод: сила кейса, confidence, что проверять вручную. |
| functions | Таблица RUNTIME_FUNCTION + unwind summary для каждой функции. | Увидеть покрытие функций, unwind RVA, handler, chain, anomalies и ошибки разбора. |

# 4. Паспорт файла и воспроизводимость

Блок report_metadata нужен не для красоты. Он отвечает на вопрос: можно ли потом доказать, что отчет относится именно к этому файлу и был получен именно этой версией инструмента.

| Поле | Значение | Практический смысл |
|---|---|---|
| schema_version | Версия JSON-контракта, сейчас 1.1.0. | Backend, тесты и фронт понимают, какой набор полей ожидать. |
| tool_version | Версия EH/Unwind Analyzer. | Позволяет отличать отчеты разных версий проекта. |
| generated_at_utc | Время генерации в UTC. | Нужно для истории анализов и сравнения запусков. |
| input_sha256 | SHA-256 анализируемого файла. | Главный идентификатор файла; лучше имени, потому что имя можно поменять. |
| input_size | Размер файла. | Дополнительная проверка идентичности и контекст для PE-валидации. |
| command_line | Команда запуска. | Показывает, были ли включены --heuristics, --dynamic, --strict-abi и другие опции. |
| build_compiler / build_platform / build_mode | Информация о сборке. | Можно понять, чем собран анализатор и в каком режиме. |
| strict_abi | Был ли включен строгий ABI-режим. | Важно: в строгом режиме появляются дополнительные ABI warnings. |
| follow_chain / max_chain_depth | Следовал ли анализатор chained unwind info и с каким лимитом. | Объясняет, почему chain учитывался или не учитывался. |

# 5. PE-карта: image, Exception Directory и секции

Этот слой отчета отвечает за адресную карту. Без него нельзя корректно читать .pdata и .xdata, потому что почти все важные ссылки в PE идут через RVA, а реальный файл читается через file offset.

| Что показывает | Какие поля | Зачем это нужно реверсеру |
|---|---|---|
| Границы image | image_base, size_of_image, size_of_headers | Понять, какие RVA допустимы и где заканчивается образ в памяти. |
| Выравнивание | section_alignment, file_alignment | Проверить корректность секций и подозрительные нарушения layout. |
| Exception Directory | exception_directory.rva, exception_directory.size | Найти таблицу RUNTIME_FUNCTION, с которой начинается анализ x64 unwind. |

| Что показывает | Какие поля | Зачем это нужно реверсеру |
|---|---|---|
| Секции | name, virtual_address, virtual_size, raw_offset, raw_size, characteristics, readable, writable, executable | Понять, где лежит код, где metadata, где writable/executable области и почему размещение подозрительно или нормально. |
| PE validation | section_raw_oob, section_virtual_oob, overlaps, misaligned, exception_directory_oob | Понять, не ломается ли базовый перевод RVA -> file offset и не повреждена ли PE-структура. |

Практический вывод из этого блока: можно сразу увидеть, лежит ли .pdata/.xdata в ожидаемых read-only metadata-секциях, нет ли секций с выходом за файл, нет ли пересечений, и можно ли вообще доверять дальнейшему RVA-маппингу.

# 6. Таблица функций RUNTIME_FUNCTION

Блок functions является сердцем статического отчета. Каждая запись соответствует одной записи RUNTIME_FUNCTION из Exception Directory. В Windows x64 эти записи связывают диапазон кода функции с UNWIND_INFO.

| Поле | Что означает | Что можно понять |
|---|---|---|
| index | Номер записи в таблице. | Удобно ссылаться на конкретную функцию из findings и HTML-таблицы. |
| begin_rva | Начало диапазона функции. | Где начинается код функции в терминах RVA. |
| end_rva | Конец диапазона функции. | Где функция заканчивается; диапазон трактуется как [begin, end). |
| size | Размер функции в байтах. | Можно искать слишком маленькие/большие/нетипичные диапазоны. |
| begin_section | Секция, куда попадает begin_rva. | Обычно ожидается .text или другая executable-секция. |
| unwind_rva | Адрес UNWIND_INFO. | Главная ссылка из .pdata в .xdata/metadata. |
| unwind_section | Секция, где лежит UNWIND_INFO. | Обычно ожидается .xdata, .rdata или metadata-секция, не executable и не writable. |
| unwind_ok | Успешно ли разобран UNWIND_INFO. | Если false, есть структурная проблема: truncation, bad opcode, tail error и т.п. |
| unwind_error | Текст ошибки разбора. | Показывает конкретную причину, почему запись не разобралась. |
| unwind | Вложенная сводка UNWIND_INFO. | Версия, flags, prolog, stack, frame, handler, chain, anomalies. |
| comment | Короткая текстовая сводка. | Удобно для Ghidra/CSV: один компактный комментарий по функции. |

Практический вывод: по таблице functions можно построить карту “какая функция каким unwind metadata описана”, увидеть функции с ошибками разбора, функции с handler-ами, функции с chain и функции, чья metadata лежит в подозрительных секциях.

# 7. Что показывает UNWIND_INFO-сводка

Вложенный блок unwind показывает не все сырые UWOP-операции в JSON, а сводное состояние UNWIND_INFO, рассчитанное парсером. Он нужен, чтобы быстро понять роль unwind-записи без ручного чтения байтов .xdata.

| Поле | Что означает | Как интерпретировать |
|---|---|---|
| version | Версия UNWIND_INFO: проект поддерживает v1 и v2. | Неожиданная версия является проблемой разбора. |
| flags | Битовая маска EHANDLER/UHANDLER/CHAININFO. | Показывает, есть ли exception/unwind handler или chained info. |

| Поле | Что означает | Как интерпретировать |
|---|---|---|
| prolog_size | Размер пролога, который описывает metadata. | Важно для ручной сверки с кодом, но проект сам не дизассемблирует пролог. |
| count_codes | Количество UNWIND_CODE slots. | Показывает объем массива unwind-кодов; учитывает multi-slot операции и padding при разборе. |
| fixed_stack_bytes | Оценка фиксированного изменения стека. | Сумма push/alloc/machframe эффектов, полезна для общей картины stack frame. |
| frame_reg / frame_off | Frame register и смещение frame pointer. | Показывает, используется ли frame pointer и как он задан. |
| has_handler / handler_rva / handler_section | Наличие handler-a, его RVA и секция. | Можно увидеть функции, участвующие в exception handling, и подозрительное размещение handler-a. |
| has_chain / chain | Наличие chained RUNTIME_FUNCTION-like entry. | Показывает переиспользование/связь unwind-записей и возможные chain-проблемы. |
| anomalies / anomalies_text | Битовая маска и текст аномалий. | Быстрая диагностика: trunc, op, arg, order, alignment, handler_oob, chain_invalid и др. |

Важно: отчет не показывает в JSON полный список каждой UWOP-операции как отдельный массив. Детальный список UWOP можно увидеть через консольный режим --dump --ops. JSON/HTML дают сводку и findings, а не пошаговую расшифровку каждой операции.

# 8. Эвристический блок: score, suspicion, confidence

Блок heuristic_analysis появляется, когда запуск включал --heuristics. Он переводит структурные наблюдения в понятную оценку подозрительности. Это не нейросетевая вероятность и не доказательство вредоносности. Это прозрачная инженерная triage-оценка по правилам.

| Поле | Что показывает | Как использовать |
|---|---|---|
| score | Итоговый счет 0..100. | Быстро понять уровень подозрительности, но не читать его как “процент вредоносности”. |
| suspicion | Нормированная подозрительность 0..1. | Машинно удобно для сортировки и фронта. |
| confidence | Инженерная уверенность вывода. | Показывает, насколько evidence устойчиво; может снижаться weakening rules. |
| requires_manual_review | Нужно ли ручное подтверждение. | Если true, отчет прямо говорит: окончательный вывод должен сделать человек. |
| verdict | no_strong_static_anomalies / low / moderate / high suspicion. | Краткий итог статического слоя. |
| evidence_level | none, weak, medium, strong. | Сила статических доказательств. |
| weak/medium/strong_finding_count | Сколько findings разной силы найдено. | Можно понять, на чем держится score. |
| evidence_families | Какие семейства признаков участвуют. | Показывает, это один тип сигнала или несколько независимых слоев. |
| weakening_rules_applied | Правила снижения уверенности. | Показывает доброкачественные объяснения или слабость evidence. |

Практический вывод: этот блок позволяет ранжировать файлы и понять, где есть сильные структурные проблемы. Он полезен для triage: какие файлы смотреть первыми, какие findings подтверждать в Ghidra, где нужна динамика.

# 9. Какие счетчики аномалий есть

Heuristic analysis хранит не только итоговый score, но и отдельные счетчики. Это важно: можно понять не просто “подозрительно”, а из-за какого слоя возникла подозрительность.

| Группа | Поля | Что означают |
|---|---|---|
| PE/section integrity | pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlaps, pe_section_virtual_overlaps, pe_section_misaligned, pe_exception_directory_oob | Проблемы базовой PE-разметки, секций и Exception Directory. Если они есть, дальнейший адресный анализ может быть менее надежным. |
| RUNTIME_FUNCTION table | rf_invalid_ranges, rf_oob, rf_unwind_oob, rf_unsorted, rf_overlaps, rf_duplicate_begins | Проблемы .pdata: неверные диапазоны, выходы за image, неотсортированность, пересечения, повторные BeginAddress. |
| UNWIND_INFO integrity | unwind_parse_fail, unwind_anomalies, unwind_abi_warnings | Ошибки разбора .xdata, странные opcode/operands/order/alignment/chain/ABI детали. |
| Metadata placement | unwind_unusual_section, unwind_exec_section, unwind_writable_section | UNWIND_INFO лежит в нетипичной, executable или writable секции. |
| Handler placement | handler_nonexec_section, handler_writable_section | Handler RVA указывает в non-executable/unknown или writable область. |
| Reuse/dominance | unwind_reuse_ratio, unwind_unique_ratio, handler_count, handler_reuse_ratio | Насколько много функций делит одну unwind-запись или один handler; помогает заметить dispatcher-style схемы. |

# 10. Findings: самая важная часть для объяснения

Finding - это конкретное обнаруженное наблюдение. Хороший отчет не должен просто писать “аномалия”. Поэтому в текущем JSON каждый finding содержит код, семейство, severity, affected_count, representative location и пояснительные поля rule/observed/expected/impact.

| Поле finding | Что показывает | Зачем нужно |
|---|---|---|
| code | Машинный код признака: например rf_overlaps или handler_nonexec_section. | Стабильный идентификатор для БД, фильтров и тестов. |
| family | Семейство: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance. | Показывает слой проблемы. |
| severity | info, weak, medium, high, critical. | Позволяет сортировать findings по важности. |
| message | Короткое описание. | Человеческое объяснение признака. |
| affected_count | Сколько объектов затронуто. | Одна проблема или массовый паттерн. |
| representative_rva | Показательный RVA. | Куда идти в PE/Ghidra для ручной проверки. |
| representative_section | Секция этого RVA. | Понять контекст: .text, .pdata, .xdata, .rdata и т.д. |
| representative_file_offset | File offset, если RVA мапится. | Открыть место в hex viewer. |
| representative_section_permissions | Права секции R/W/X. | Понять, почему placement нормальный или подозрительный. |
| related_function_* | Связанная функция, если finding можно сопоставить с RUNTIME_FUNCTION. | Переход от признака к конкретной функции. |
| rule | Какое правило должно выполняться. | Объясняет критерий проверки. |
| observed | Что реально найдено. | Фактическое evidence. |
| expected | Что ожидалось. | Нормальное состояние для сравнения. |
| impact | Почему это влияет на анализ. | Объясняет риск: broken unwind, ambiguous stack walking, suspicious placement и т.д. |

Практический вывод: findings можно использовать как checklist для ручной проверки. У finding есть адрес, секция, причина, ожидаемое состояние и влияние. Это уже не просто “score”, а объяснимая evidence-карточка.

# 11. Семейства признаков

| Семейство | Что проверяет | Какие вопросы закрывает |
|---|---|---|
| table_integrity | Целостность Exception Directory и RUNTIME_FUNCTION table. | Отсортирована ли .pdata? Нет ли пересечений? Все ли RVA внутри SizeOfImage? |
| unwind_integrity | Корректность UNWIND_INFO и unwind-кодов. | Разбирается ли .xdata? Нет ли bad opcode, bad operand, chain loop, truncation, alignment issue? |
| metadata_placement | Где физически/виртуально лежит unwind metadata. | Лежит ли metadata в read-only секции или подозрительно в executable/writable области? |
| unwind_reuse | Повторное использование одной UNWIND_INFO многими функциями. | Это нормальный compiler pattern или подозрительная централизованная разметка? |
| handler_dominance | Повторное использование handler target. | Один handler обслуживает слишком много функций? Это runtime helper или dispatcher-style схема? |

# 12. Weakening rules: почему score может ослабляться

Отчет не пытается любой странный факт сразу объявить вредоносным. Поэтому есть weakening_rules_applied. Это правила, которые снижают уверенность, если evidence одинокое, слабое, объяснимое доброкачественным layout-ом или требует ручного подтверждения.

- single_signal_only - есть только один тип сигнала, поэтому нельзя делать сильный вывод без подтверждения.
- small_sample_size - слишком мало функций/metadata, чтобы уверенно говорить о массовом паттерне.
- placement_only_signal - странное размещение metadata само по себе может быть редким, но доброкачественным layout-ом.
- single_family_only - признаки относятся к одному семейству, нет независимой корреляции.

Практический смысл: отчет показывает не только причины подозрения, но и причины осторожности. Это полезно для защиты проекта: система не переобещает, а честно отделяет сильные выводы от требующих проверки.

# 13. Assessment: итоговая оценка

Блок assessment собирает статическую эвристику и динамические сигналы в общий вывод. Он нужен, чтобы не заставлять пользователя вручную складывать score, dynamic events и weakening rules.

| Поле | Что показывает | Как читать |
|---|---|---|
| final_verdict | Итоговый вывод: от no_strong_correlated_evidence до high_suspicion_correlated. | Короткий статус всего отчета. |
| final_confidence | Итоговая confidence 0..1. | Не вероятность вредоносности, а уверенность evidence-модели. |
| static_evidence_level | Сила статических доказательств. | Что дала статика: none/weak/medium/strong. |
| runtime_evidence_level | Сила динамических доказательств. | Что дала динамика, если запускалась. |
| case_strength | Сила кейса: no_case, weak_case, medium_case, strong_unconfirmed_case, medium_correlated_case, strong_correlated_case. | Насколько связная доказательная картина. |
| static_dynamic_correlation | Есть ли корреляция между статикой и динамикой. | Важно: динамика может подтвердить или ослабить статическое подозрение. |

| Поле | Что показывает | Как читать |
|---|---|---|
| dynamic_signal_count | Количество динамических сигналов. | Сколько runtime-событий повлияло на assessment. |
| weakening_rule_count | Сколько правил ослабления применено. | Почему вывод может быть осторожнее, чем raw score. |
| manual_validation_steps | План ручной проверки. | Что реверсеру нужно открыть в PE viewer/Ghidra/debugger после отчета. |

# 14. Динамический отчет

Блок dynamic_validation появляется, когда запускался --dynamic. Он относится только к Windows dynamic validation: target запускается под Windows Debug API. Это уже не чистая статика: файл реально исполняется, поэтому для неизвестных файлов нужен изолированный стенд или VM.

| Группа полей | Поля | Что можно понять |
|---|---|---|
| Статус запуска | enabled, ran, ok, timed_out, target_exited, target_exit_code, timeout_ms, trace_steps | Запускалась ли динамика, завершился ли target, был ли timeout, какие лимиты использовались. |
| Полнота инструментирования | instrumentation_complete, modules_observed_peak, module_refresh_failures, module_list_truncations | Насколько полно отладчик видел модули и не было ли технических ограничений наблюдения. |
| Breakpoints | unwind_handlers_discovered, breakpoint_set_attempts, breakpoint_set_failures, breakpoint_capacity_drops, pending_rearm_failures, breakpoint_rearm_failures, unwind_handler_bp_set | Удалось ли поставить breakpoints на handler targets и не потерялись ли события из-за лимитов. |
| Исключения | exceptions_seen, first_chance_seen, second_chance_seen | Были ли exceptions во время исполнения и на какой фазе они наблюдались. |
| Handler activity | unwind_handler_hits, veh_registrations, veh_hits, vch_registrations, vch_hits, uef_set_calls, uef_hits | Регистрировались ли VEH/VCH/UEF и реально ли управление доходило до handlers. |
| Runtime function tables | rtl_add_function_table_calls, rtl_install_function_table_cb_calls | Были ли runtime-регистрации function table, важные для JIT/runtime-generated code. |
| Надежность журнала | dynamic_signal_count, dropped_events, details | Сколько сигналов собрано и не были ли события потеряны. |
| События и trace | events, trace | Список runtime events и ограниченный RIP trace после handler hit. |

Важно: текущая динамика фиксирует факт runtime-событий и адреса/аргументы событий, но не снимает полный crash dump и не делает пошаговый RtlVirtualUnwind с восстановлением регистров из памяти стека.

# 15. Dynamic events: что внутри события

| Поле event | Что означает | Зачем нужно |
|---|---|---|
| type | Тип события: exception, handler hit, VEH/VCH/UEF registration/hit, runtime table API и т.п. | Понять, что именно произошло во время исполнения. |
| module | Модуль, к которому относится событие. | Отличить событие основного exe от DLL/runtime. |
| thread_id | Поток. | Понять, в каком thread произошло событие. |
| exception_code | Код исключения, если применимо. | Отличить access violation, breakpoint, custom exception и т.д. |
| first_chance | First-chance или second-chance/n-a. | Понять фазу обработки exception. |

| Поле event | Что означает | Зачем нужно |
|---|---|---|
| address | Адрес события. | Точка, куда можно идти в debugger/Ghidra. |
| arg0..arg3 | Аргументы события. | Для API hooks здесь могут быть адреса таблиц, count, base и другие захваченные параметры. |

# 16. HTML-отчет: что видно человеку

HTML-отчет - это self-contained dashboard без сетевых ресурсов. Он нужен для чтения человеком. В нем есть карточки, таблицы, итоговая оценка, план ручной проверки, статические признаки, детали анализа, динамическая проверка и таблица RUNTIME_FUNCTION.

| Раздел HTML | Что показывает |
|---|---|
| Верхние карточки | Количество функций, количество findings, статический score, итоговый вывод, статика/динамика, dynamic signals, confidence, статус динамики. |
| Метаданные | Когда сформирован отчет, версия инструмента/schema, SHA-256, сборка, параметры анализа, размер файла, команда запуска. |
| Образ PE | ImageBase, SizeOfImage, SizeOfHeaders, Exception Directory, alignment. |
| Структурная проверка | Счетчики raw/virtual OOB, overlaps, misalignment, exception directory OOB. |
| Секции | Таблица секций с virtual/raw размерами и правами. |
| Итоговая оценка | Вывод, сила доказательств, confidence, static/runtime evidence, корреляция слоев. |
| План ручной проверки | Пункты, что открыть и проверить вручную после отчета. |
| Статические признаки | Findings с severity, code, family, affected count, RVA, section и русскими пояснениями. |
| Динамическая проверка | Exceptions, handler hits, runtime table calls, dropped events, instrumentation completeness, event table. |
| Таблица RUNTIME_FUNCTION | Begin, End, Size, code section, Unwind RVA, unwind section, status, version/flags, stack, handler, anomalies/error. |

# 17. CSV и Ghidra CSV

CSV-форматы нужны для внешних инструментов. Полный CSV удобен для таблиц и быстрых фильтров, а Ghidra CSV - для переноса function ranges и unwind-комментариев в Ghidra.

| Формат | Поля | Что дает |
|---|---|---|
| CSV | index, begin_rva, end_rva, size, unwind_rva, unwind_ver, unwind_flags, prolog_size, fixed_stack, frame_reg, frame_off, handler_rva, has_chain, anomalies, parse_error | Плоская таблица по функциям. Можно сортировать по stack, handler, anomalies, parse_error. |
| Ghidra CSV | begin_rva, end_rva, name, comment | Ghidra получает диапазоны функций, имя вида sub_XXXXXXXX и комментарий с UNWIND summary. |

# 18. Какие практические вопросы закрывает отчет

1. Есть ли в файле Exception Directory и сколько RUNTIME_FUNCTION записей удалось прочитать?
2. Какие диапазоны функций описаны в .pdata и где лежит их UNWIND_INFO?
3. У каких функций UNWIND_INFO не разбирается и почему именно?
4. Какие функции имеют EHANDLER/UHANDLER/CHAININFO?
5. Куда указывает handler RVA и в какой секции он находится?
6. Лежит ли unwind metadata в нормальной metadata-секции или в executable/writable области?
7. Есть ли пересечения, неотсортированность, duplicate BeginAddress или выходы RVA за SizeOfImage?
8. Есть ли массовое повторное использование одного UnwindInfoAddress или одного handler-а?

9. Насколько сильны признаки: weak, medium, high, critical?
10. Какие признаки поддерживают друг друга, а какие ослабляются benign-объяснениями?
11. Были ли runtime-события: exceptions, handler hits, VEH/VCH/UEF, RtlAddFunctionTable?
12. Есть ли корреляция между статическими подозрениями и динамическими событиями?
13. Что нужно проверить вручную в Ghidra, PE viewer или debugger после отчета?

# 19. Что отчет не показывает

Этот раздел важен, потому что на защите нельзя переобещать. Текущий отчет сильный как структурный и эвристический анализ EH/Unwind metadata, но у него есть понятные границы.

| Не показывает | Почему это важно |
|---|---|
| Не дизассемблирует пролог .text и не сравнивает каждую UWOP с конкретной инструкцией. | Поэтому он не пишет “UWOP_PUSH_NONVOL RBX matches push rbx”. Это улучшение будущего prolog-vs-unwind validator. |
| Не выполняет пошаговую симуляцию RtlVirtualUnwind с реальными RSP/RIP/registers. | Для этого нужны ControlPc, register context и память стека или crash dump/live process context. |
| Не парсит универсально language-specific handler data. | Формат зависит от handler-а: C SEH, C++ EH, custom runtime и т.д. Универсальный parser был бы методологически неверным. |
| Не доказывает вредоносность файла. | Score - triage aid, а не юридическое/криптографическое доказательство malware. |
| Не гарантирует отсутствие аномалий вне EH/Unwind области. | Проект специализирован на PE x64 EH/Unwind metadata, а не на полном malware analysis всего файла. |
| Dynamic mode не безопасен для неизвестных файлов без VM. | Динамика исполняет target, поэтому это отдельная trust boundary. |

# 20. Как читать отчет по шагам

1. Сначала открыть верхние карточки HTML или assessment в JSON: понять общий verdict, confidence, static/runtime evidence.
2. Проверить report_metadata: SHA-256, команда запуска, версия schema, был ли --heuristics и --dynamic.
3. Посмотреть PE-карту: ImageBase, SizeOfImage, Exception Directory, секции и права секций.
4. Проверить pe_validation: нет ли грубых проблем секций и Exception Directory.
5. Перейти к functions: сколько RUNTIME_FUNCTION записей, какие begin/end/unwind RVA, где лежит metadata.
6. Отфильтровать функции с unwind_ok=false, anomalies_text != 0, handler_rva, has_chain=true.
7. Открыть heuristic_analysis: score, confidence, evidence_families, counters.
8. Разобрать findings: смотреть code, severity, representative_rva, observed, expected, impact.
9. Если есть weakening rules, понять, почему вывод ослаблен и что может быть benign-объяснением.
10. Если запускалась динамика, проверить exceptions, handler hits, VEH/VCH/UEF, runtime table API, dropped_events.
11. Прочитать manual_validation_steps и выполнить их в Ghidra/PE viewer/debugger.

# 21. Как использовать отчет с БД и фронтом

JSON уже является удобной точкой интеграции. Backend может принимать бинарник, запускать ядро, сохранять report.json и раскладывать его по таблицам. Ядро при этом менять не обязательно: оно выступает producer-ом отчетов.

| Сущность БД | Откуда брать |
|---|---|
| file | input, report_metadata.input_sha256, input_size, generated_at_utc, command_line |
| pe_image | image_base, size_of_image, size_of_headers, alignments, exception_directory |
| sections | sections[] |
| functions | functions[] begin/end/size/sections/unwind_rva/unwind_ok |
| unwind_summary | functions[].unwind |
| findings | heuristic_analysis.findings[] |
| assessment | assessment |
| dynamic_run | dynamic_validation status/counters |
| dynamic_events | dynamic_validation.events[] |

Фронт может строить страницы: overview, PE map, sections, function table, findings, dynamic events, comparison by SHA-256. Самое полезное для сравнения версий файла - SHA-256, score, findings by code, affected_count, representative_rva и функции с изменившимся unwind_rva/anomalies.

# 22. Формулировка для защиты

“Отчет EH/Unwind Analyzer показывает не просто поля PE, а связанную evidence-картину по x64 exception/unwind metadata: паспорт файла, PE layout, секции и права, Exception Directory, таблицу RUNTIME_FUNCTION, сводку UNWIND_INFO по каждой функции, handler-ы, chained records, структурные аномалии, эвристические findings, уровень подозрительности, confidence, rules that weaken confidence, dynamic exception evidence and manual validation steps. Его цель - ускорить и стандартизировать первичный разбор EH/Unwind metadata для реверсера.”

“При этом текущий отчет честно ограничен: он не является полной симуляцией RtlVirtualUnwind и не сравнивает каждую UWOP с дизассемблированной инструкцией пролога. Эти функции обозначены как развитие проекта. Текущая версия закрывает структурный анализ и объяснимую эвристическую оценку.”

# 23. Короткая шпаргалка

| Если нужно узнать... | Где смотреть |
|---|---|
| Какой файл анализировался | report_metadata.input_sha256, input, input_size |
| Где .pdata / Exception Directory | exception_directory.rva / size |
| Какие секции есть и какие у них права | sections[] или раздел “Секции” в HTML |
| Какие функции описаны unwind metadata | functions[] |
| Где лежит UNWIND_INFO конкретной функции | functions[i].unwind_rva, unwind_section |
| Есть ли handler | functions[i].unwind.has_handler, handler_rva, handler_section |
| Есть ли chain | functions[i].unwind.has_chain, chain |
| Почему функция подозрительна | heuristic_analysis.findings[] и related_function_* |
| Насколько сильны признаки | severity, evidence_level, score, confidence |
| Что делать вручную | assessment.manual_validation_steps |
| Что произошло в динамике | dynamic_validation.events, counters, trace |

# 24. Итог

Отчет проекта показывает важную для реверсера картину: как PE x64 файл описывает свои функции через .pdata и .xdata, насколько эти описания структурно корректны, где находятся handler-ы и metadata, какие признаки выглядят подозрительно, как сильна evidence-модель и что нужно проверить вручную. Его главная ценность - объяснимая структурная карта и triage по EH/Unwind metadata.

Самый правильный способ описать отчет: это не “черный ящик с вердиктом”, а техническое досье. Оно дает адреса, секции, file offsets, rules, observed/expected/impact, severity, dynamic counters и manual steps. После него реверсер быстрее понимает, куда идти в Ghidra или debugger и какие гипотезы проверять.

# 25. Что проект делает сам, а что интерпретирует специалист

Для защиты это один из самых важных разделов, потому что он правильно формулирует роль проекта. EH/Unwind Analyzer не является просто “дампером полей”, который бездумно печатает содержимое PE. Он выполняет автоматизированный структурный анализ: читает данные, проверяет правила формата, связывает разные структуры между собой, выделяет findings и формирует первичную оценку подозрительности. Но он также не является полностью автоматическим реверсером, который сам доказывает смысл каждой инструкции и принимает окончательное экспертное решение вместо человека.

Самая точная формулировка такая: проект автоматически собирает и проверяет evidence по PE x64 EH/Unwind metadata, а специалист-реверсер на основе этого evidence делает глубокую интерпретацию в контексте конкретного компилятора, runtime, защитного механизма, packer-a, JIT-сценария или ручной модификации файла.

## 25.1. Почему это не просто “выдать данные”

Сырые данные - это когда инструмент просто показывает числа: RVA, размер секции, flags, bytes. В таком режиме человеку все равно приходится самому понимать, корректно ли это, где связь между таблицами, какие адреса опасны, какие секции подозрительны, а какие варианты нормальны для компилятора. Текущий проект делает больше.

| Уровень | Что было бы простым дампом | Что делает проект |
|---|---|---|
| Извлечение | Напечатать поля PE и .pdata. | Читает PE, строит карту секций, переводит RVA в file offset, читает Exception Directory и RUNTIME_FUNCTION. |
| Декодирование | Показать байты .xdata. | Разбирает UNWIND_INFO: version, flags, prolog_size, CountOfCodes, stack effect, handler, chain, anomalies. |
| Проверка | Оставить человеку искать ошибки. | Автоматически проверяет OOB, overlaps, invalid ranges, bad opcodes, tail layout, chain problems, handler placement и metadata placement. |
| Связи | Поля лежат отдельно. | Связывает finding с representative RVA, section, file offset и related function. |
| Оценка | Нет вывода. | Выдает score, suspicion, confidence, verdict, evidence level, severity, weakening rules и manual validation steps. |
| Отчетность | Только терминал. | Формирует JSON для БД, HTML для человека, CSV для таблиц и Ghidra CSV для дизассемблера. |

## 25.2. Что проект реально делает автоматически

1. Строит адресную карту PE: секции, raw/virtual размеры, права R/W/X, ImageBase, SizeOfImage, Exception Directory.
2. Проверяет базовую целостность PE layout: выходы секций за файл/image, пересечения, misalignment, exception directory out-of-bounds.
3. Читает таблицу RUNTIME_FUNCTION как карту функций Windows x64: begin RVA, end RVA, unwind info RVA.
4. Проверяет таблицу функций: invalid ranges, OOB, unsorted entries, overlaps, duplicate BeginAddress.
5. Разбирает UNWIND_INFO: version, flags, prolog size, count of codes, frame register, fixed stack, handler, chained info.
6. Учитывает сложные детали формата: multi-slot UWOP operations, padding до хвоста, handler/chained tail, chain depth и loop detection.
7. Фиксирует ошибки разбора per function, чтобы человек видел не просто “не удалось”, а конкретную причину.

8. Строит эвристические findings: код признака, семейство, severity, affected count, representative RVA, section, rule, observed, expected, impact.
9. Считает score/confidence/verdict и отдельно показывает weakening rules, если evidence недостаточно сильное или может иметь benign-объяснение.
10. При динамическом запуске на Windows собирает runtime evidence: exceptions, first/second chance, handler hits, VEH/VCH/UEF activity, RtlAddFunctionTable/RtlInstallFunctionTableCallback calls, trace and dropped events.

## 25.3. Что остается специалисту-реверсеру

Специалист нужен не потому, что проект бесполезен, а потому что EH/Unwind metadata живет в большом контексте: compiler output, runtime, оптимизации, language-specific handlers, JIT, packers, protectors, unusual but legitimate layouts. Проект показывает структурные признаки и адреса, но эксперт решает, что они означают в конкретном бинарнике.

- Понять, является ли странное размещение metadata ошибкой, защитной техникой, особенностью компилятора или допустимым runtime/JIT-сценарием.
- Открыть representative RVA в Ghidra/IDA/hex viewer и проверить surrounding context.
- Сравнить функции, которые ссылаются на один UnwindInfoAddress или один handler, и решить, похоже ли это на normal runtime helper или central dispatcher.
- Проверить handler target: это реальный обработчик, thunk, import stub, non-code blob или переход в защитный dispatcher.
- При необходимости запустить динамику в VM и сопоставить runtime events со статическими findings.
- Сделать финальный вывод: benign anomaly, compiler/runtime feature, corrupted metadata, obfuscation, packer/protector или malicious manipulation.

## 25.4. Чего проект пока не делает сам

Это нужно произносить честно. Текущий проект закрывает структурный анализ и triage, но пока не закрывает глубокую семантическую проверку machine code против unwind metadata.

| Пока не делает | Что это означает |
|---|---|
| Не дизассемблирует пролог функции. | Проект знает prolog_size из UNWIND_INFO, но не показывает реальные инструкции вида push rbx / sub rsp, 40h из .text. |
| Не сравнивает каждую UWOP с инструкцией. | Он не пишет “UWOP_PUSH_NONVOL RBX matches push rbx” и не создает finding “RBX описан metadata, но не найден в прологе”. |
| Не симулирует RtlVirtualUnwind пошагово. | Он не берет ControlPc, RSP, память стека и не показывает восстановление caller RIP/registers. |
| Не парсит универсально language-specific handler data. | Формат зависит от конкретного handler-а: C SEH, C++ EH, custom runtime. Универсально разбирать это нельзя без знания handler kind. |
| Не заменяет итоговое экспертное решение. | Score и verdict - triage, а не окончательный судебный вывод о вредоносности или типе защиты. |

## 25.5. Как правильно сказать на защите

> “Проект не является простым парсером и не ограничивается печатью полей. Он автоматически извлекает EH/Unwind metadata из PE x64, проверяет структурные инварианты, связывает RUNTIME_FUNCTION с UNWIND_INFO, handler-ами, секциями и file offsets, выявляет аномалии, оценивает их тяжесть, формирует findings и показывает, что нужно проверить вручную. То есть он автоматизирует первичный структурный анализ и triage, которые реверсер обычно делал бы руками.”

> “При этом проект не претендует на полную замену реверсера. Он не делает глубокое дизассемблирование пролога и не симулирует RtlVirtualUnwind по реальному стеку. Поэтому окончательная интерпретация остается за специалистом: он берет evidence из отчета, открывает связанные RVA в Ghidra/debugger и решает, является ли наблюдение нормальным compiler/runtime behavior, повреждением, обфускацией или признаком вмешательства.”

## 25.6. Почему такая модель полезна

На практике реверсеру редко нужен инструмент, который делает вид, что “сам все понял”. Гораздо полезнее инструмент, который быстро и одинаково для всех файлов строит карту evidence, не пропускает типовые ошибки формата, показывает точные адреса и объясняет, почему конкретное место требует внимания. Именно это делает текущий отчет.

- Он экономит время: не нужно вручную искать Exception Directory, мапить каждый RVA и читать каждый UNWIND_INFO с нуля.
- Он снижает риск человеческих ошибок: CountOfCodes, padding, chained tail, handler RVA и OOB проверяются автоматически.
- Он стандартизирует анализ: разные файлы получают один формат JSON/HTML/CSV и одинаковые критерии findings.
- Он помогает обучению: студент видит, какие поля важны, как они связаны и какие manual steps делать дальше.
- Он удобен для развития продукта: JSON можно сохранять в БД, сравнивать отчеты, строить frontend и накапливать corpus.

## 25.7. Короткая финальная формула

Проект выдает не просто данные, а структурированное evidence. Он сам выполняет извлечение, проверку, связывание, классификацию признаков и первичную оценку. Специалист на основе этих данных выполняет глубокую семантическую интерпретацию: смотрит код, runtime-контекст, compiler/runtime объяснения и принимает окончательное решение.

Именно поэтому корректное название роли проекта: инструмент автоматизированного структурного анализа и triage PE x64 EH/Unwind metadata. Он не “автоматический реверсер”, но он сильно ускоряет и дисциплинирует работу реверсера.